MathDevMCP

Final Release Report for an Industrial Mathematical Development Agent

Chak Wong and collaborators

April 29, 2026

Contents

Part I

Release Claim

Chapter 1

Executive Summary

MathDevMCP is now a working internal release candidate for document-grounded mathematical development. It provides a Python library, command-line tools, MCP tool wrappers, benchmark gates, release profiles, backend diagnostics, and privacy-preserving private-corpus validation. The product is designed for colleagues who work across long LaTeX documents, mathematical code, proofs, derivations, algorithms, and review workflows where unsupported claims are expensive.

The release claim is deliberately narrow and evidence based. MathDevMCP does not claim to prove arbitrary mathematical papers correct. It gives agents and humans structured access to local documents, labels, code, parser evidence, typed obligations, AST operation evidence, backend diagnostics, and release gates. It helps colleagues find the right mathematical context, compare documented claims to implementations, detect missing implementation terms, route proof obligations, and abstain when evidence is not certifying.

The current full release profile is configured to require benchmark evidence, parser policy, governance validation, release-corpus metadata, LeanDojo backend environment evidence, LaTeXML validation, and a private-corpus manifest. The private-corpus evidence used for this report is an external sanitized corpus generated outside the repository. Its purpose is to validate the privacy and release-gate machinery without committing private department documents. A real department manifest can be substituted through the same environment variable.

Release status. With an external sanitized private manifest supplied, the full profile has no blocking findings. During report generation the only remaining caveat is the dirty working tree caused by ongoing release edits.

Chapter 2

Product Scope

MathDevMCP supports five colleague-facing activities:

1. locating mathematical context in LaTeX source trees;
2. extracting labeled equation, theorem, assumption, and paragraph neighborhoods with provenance;
3. comparing document claims with Python implementations;
4. auditing derivation obligations with parser, typed, shape, numeric, and backend evidence;
5. producing repeatable release evidence across base, backend, LaTeXML, private-corpus, and full profiles.

The product is intentionally local-first. Source documents and code remain on the user's machine. Optional tools such as LaTeXML, Lean, LeanDojo, Sage, and SymPy are detected at runtime and reported through structured diagnostics. LeanDojo remains in the isolated `mathdevmcp-backends` environment so its dependency stack does not destabilize the main working environment.

2.1 What is included

The release includes:

- a LaTeX indexer with label, environment, and context extraction;
- code and document search utilities;
- code–document consistency checks;
- derivation-step checks with explicit *unverified* and *mismatch* outcomes;

- proof obligation and proof-audit workflows;
- typed and dimensional diagnostic workflows;
- AST operation extraction for mathematical implementation patterns;
- parser benchmark and parser policy selection;
- release corpus manifest validation;
- private-corpus validation with redacted output;
- release-readiness profiles;
- MCP facade and FastMCP server wrappers;
- operator, deployment, release, security, and maintainer documentation.

2.2 What is not claimed

MathDevMCP does not replace expert mathematical review. A parser hit is not a proof. A shape diagnostic is not a theorem. A Lean skeleton is not a certificate until direct Lean checking accepts it without placeholders. A numeric diagnostic can refute a false claim or suggest a failure mode, but it does not establish a general theorem. The product's value is that it preserves these distinctions in machine-readable reports.

Chapter 3

Release Profiles

The release profiles are:

base Public benchmarks, parser policy, governance, release corpus, and normal doctor evidence.

backend Base evidence plus isolated LeanDojo backend environment validation.

latexml Base evidence plus strict LaTeXML parser validation.

private-corpus Base evidence plus an external private or sanitized private manifest.

full All of the above.

The profile split is important because colleagues may use the base product without every optional backend, while release managers can demand strict evidence before a broader deployment. Optional evidence is not silently ignored: strict profiles turn missing evidence into blocking findings.

3.1 Full profile evidence

Listing 3.1: Full profile release-readiness summary

```
Command: PYTHONPATH=src python -m mathdevmcp.cli release-readiness --
   root "$PWD" --profile full
Status: ready_with_caveats
Reason: Release gates passed with documented caveats.
Blockers: []
Caveats: ['dirty_worktree']
Git commit: 2f49963
Dirty worktree: True
```

3.2 Doctor evidence

Listing 3.2: Doctor capability summary

```
Command: PYTHONPATH=src python -m mathdevmcp.cli doctor
Overall ok: True
Python: 3.11.15 at /home/chakwong/miniconda3/envs/tfgpu/bin/python
LaTeXML: available (latexml (LaTeXML version 0.8.6))
Pandoc: available (pandoc 2.9.2.1)
Lean: available (Lean (version 4.20.0, x86_64-unknown-linux-gnu, commit
    77cfc4d1a4f6, Release))
Sage: available (SageMath version 9.5, Release Date: 2022-01-30)
LeanDojo in active env: unavailable (Python module lean_dojo is not
   importable)
SymPy: available (1.14.0)
Conflicts: 0
```

Part II

Architecture

Chapter 4

System Architecture

MathDevMCP has three layers. The first layer is a Python implementation substrate that owns parsing, indexing, consistency checks, proof-audit routing, benchmarking, release policy, and diagnostics. The second layer is a CLI and MCP facade that presents stable tools to users and agents. The third layer is a workflow and documentation layer that explains how to combine tools for colleague tasks.

4.1 Implementation substrate

The Python package is organized around small report-producing functions. Each public report includes a metadata contract so that CLI, MCP, tests, and release docs can depend on stable shapes. For example, release readiness returns a `release_readiness_report`; parser benchmark returns `parser_backend_result`; private corpus validation returns `private_corpus_validation_report`. This is the main compatibility discipline for future maintainers.

4.2 CLI and MCP facades

The CLI is the reproducible interface for humans and CI. The MCP facade wraps the same library functions so an agent can call them interactively. The facade does not grant unrestricted shell access. It exposes bounded tools such as `search_latex`, `compare_label_code`, `audit_derivation_v2_label`, `benchmark_gate`, and `release_readiness`.

4.3 Backend isolation

Optional backends are treated as evidence engines. LaTeXML, Pandoc, Lean, Sage, SymPy, and LeanDojo are detected by `doctor`. LeanDojo is checked through the backend environment when configured. This avoids dependency conflicts in the main Python environment while still allowing proof-search experiments and Lean fixture validation.

Chapter 5

Core Data Contracts

Every user-facing workflow should be understood as a contract. A contract has a schema version, a name, a status or result payload, and where appropriate provenance. This is why downstream documentation, agents, and tests can interpret outputs without scraping prose.

5.1 Status vocabulary

The primary statuses are:

consistent The requested terms or structure were found in the checked context.

equivalent A scoped derivation step matched exactly after normalization or backend checking.

unverified Evidence exists, but it is not enough for certification.

mismatch Required structure is missing or contradicted.

inconclusive The tool lacks enough structure, parser support, or backend evidence to decide.

This vocabulary is central to the product. It prevents the agent from turning a nearby equation or diagnostic hint into a categorical mathematical claim.

Chapter 6

Parser Policy

MathDevMCP measures parser behavior before relying on parser output for proof audit. The current lightweight parser is selected for proof audit when it preserves expected labels and line provenance. LaTeXML is also validated in the strict LaTeXML profile, but current release routing prefers the provenance and environment structure available through the current parser.

Listing 6.1: Parser policy summary

```
Command: PYTHONPATH=src python -m mathdevmcp.cli parser-benchmark --
    root "$PWD/benchmarks/fixtures" --backend current
Policy status: selected_for_proof_audit
Selected backend: current
Labels found: 67
Environments found: 67
Align-like blocks found: 4
Provenance quality: line
Blocking findings: 0
```

Chapter 7

Benchmark Gate

The benchmark gate is the main regression control. It covers consistency, derivation, workflow, proof-audit, parser-corpus, AST-corpus, typed IR, industrial review, and release-corpus cases. The gate currently requires every benchmark case to pass; expected abstentions are counted as quality signals, not as failures.

Listing 7.1: Benchmark gate summary

```
Command: PYTHONPATH=src python -m mathdevmcp.cli benchmark-gate --root
    "$PWD"
Passed: True
Cases: 41/41
Expected abstentions: 12
Categories:
  consistency: 5/5 passed, expected abstentions 0
  derivation: 6/6 passed, expected abstentions 5
  workflow: 3/3 passed, expected abstentions 1
  proof_audit: 3/3 passed, expected abstentions 1
  proof_audit_v2: 3/3 passed, expected abstentions 1
  kalman_recursion: 2/2 passed, expected abstentions 1
  parser_corpus: 3/3 passed, expected abstentions 0
  ast_corpus: 11/11 passed, expected abstentions 0
  typed_ir: 2/2 passed, expected abstentions 2
  industrial_review: 1/1 passed, expected abstentions 1
  release_corpus: 1/1 passed, expected abstentions 0
  release_policy: 1/1 passed, expected abstentions 0
```

Part III

Colleague Workflows

Chapter 8

Workflow 1: Find Mathematical Context

A colleague begins with a topic such as a likelihood, Hamiltonian flow, Euler equation, or stability condition. The first tool is usually `search-latex`. It returns document blocks with labels and provenance so the agent can ground a response in the source document.

Listing 8.1: Search workflow output

```
Command: python -m mathdevmcp.cli search-latex Kalman likelihood --root <repo>/benchmarks/fixtures --limit 3
[
  {
    "kind": "section",
    "name": "Likelihood derivative notes",
    "file": "doc_longdoc_kalman_retrieval.tex",
    "line_start": 23,
    "line_end": 23,
    "label": null,
    "title": "Likelihood derivative notes",
    "text": "\\section{Likelihood derivative notes}",
    "block_id": "doc_longdoc_kalman_retrieval.tex:23:section:Likelihood-derivative-notes",
    "section_path": [
      "Likelihood derivative notes"
    ],
    "score": 2
  },
  {
    "kind": "equation",
    "name": "equation",
    "file": "doc_longdoc_kalman_retrieval.tex",
    "line_start": 26,
    "line_end": 31,
    "label": "eq:longdoc-score-contribution",
    "title": null,
    "text": "\\begin{equation}\\label{eq:longdoc-score-contribution}\n\\partial_i \\ell_t\n= -\\frac{1}{2}\\operatorname{tr}(S_t^{-1}\\partial_i S_t)\n+ \\frac{1}{2}v_t' S_t^{-1}(\\partial_i S_t)S_t^{-1}v_t\n- (\\partial_i v_t)'S_t^{-1}v_t.\n\\end{equation}",
    "block_id": "doc_longdoc_kalman_retrieval.tex:26:equation:eq:longdoc-score-
```

```
        contribution",
    "section_path": [
      "Likelihood derivative notes"
    ],
    "score": 2
  },
  {
    "kind": "section",
    "name": "Kalman likelihood Hessian",
    "file": "doc_realistic_kalman_hessian.tex",
    "line_start": 1,
    "line_end": 1,
    "label": null,
    "title": "Kalman likelihood Hessian",
    "text": "\\section{Kalman likelihood Hessian}",
    "block_id": "doc_realistic_kalman_hessian.tex:1:section:Kalman-likelihood-Hessian",
    "section_path": [
      "Kalman likelihood Hessian"
    ],
    "score": 2
  }
]
```

Chapter 9

Workflow 2: Read a Labeled Neighborhood

Once a label is known, the user can extract a paragraph neighborhood. This is more useful than a raw line excerpt when an agent needs nearby explanatory prose, assumptions, or notation.

Listing 9.1: Labeled neighborhood output

```
Command: python -m mathdevmcp.cli extract-latex-neighborhood eq:dept-state-space-likelihood --root <repo>/benchmarks/fixtures
{
  "file": "doc_department_state_space.tex",
  "line_start": 21,
  "line_end": 23,
  "context_start": 11,
  "context_end": 23,
  "paragraphs": [
    {
      "line_start": 11,
      "line_end": 19,
      "text": "\\begin{align}\\label{eq:dept-state-space-recursion}\nx^{\\mathrm{pred}}_t &= F_t x_{t-1} \\\\\n\\PredCov &= F_t P_{t-1}F_t' + Q_t \\\\\n\\Innov &= y_t - H_t x^{\\mathrm{pred}}_t \\\\\n\\InnovCov &= H_t \\PredCov H_t' + R_t \\\\\nK_t &= \\PredCov H_t' \\InnovCov^{-1} \\\\\nx_t &= x^{\\mathrm{pred}}_t + K_t \\Innov \\\\\nP_t &= \\PredCov - K_t H_t \\PredCov\n\\end{align}"
    },
    {
      "line_start": 21,
      "line_end": 23,
      "text": "\\begin{equation}\\label{eq:dept-state-space-likelihood}\n\\ell_t = -\\frac{1}{2}\\left(\\log\\det \\InnovCov + \\Innov' \\InnovCov^{-1}\\Innov\\right).\n\\end{equation}"
    }
  ],
  "label": "eq:dept-state-space-likelihood",
  "kind": "equation",
  "block_id": "doc_department_state_space.tex:21:equation:eq:dept-state-space-likelihood",
  "section_path": [
    "Sanitized state-space audit slice"
```

```
  ]
}
```

Chapter 10

Workflow 3: Compare Document and Code

Code–document drift is one of the highest-value release use cases. The `compare-label-code` workflow checks whether required terms from a labeled document block are present in a code file. It is not a full proof of equivalence. It is a targeted drift detector.

Listing 10.1: Code–document comparison output

```
Command: python -m mathdevmcp.cli compare-label-code eq:dept-state-space-likelihood <repo>/benchmarks/fixtures/doc_department_state_space_missing_solve.py --root <repo>/benchmarks/fixtures --required-terms logdet,solve --paragraph-context
{
  "status": "mismatch",
  "reason": "Some required document terms were not found in the code text.",
  "doc_terms": [
    "logdet",
    "solve"
  ],
  "code_terms": [
    "def",
    "kalman_filter_scan",
    "jnp",
    "lax",
    "params",
    "step",
    "carry",
    "obs",
    "x_pred",
    "p_pred",
    "innovation",
    "sign",
    "logdet",
    "linalg",
    "slogdet",
    "x_filt",
    "p_filt",
    "ll_t",
    "return",
    "scan",
    "initial_state"
```

```
  ],
  "missing_in_code": [
    "solve"
  ],
  "findings": [
    {
      "kind": "matched_term",
      "term": "logdet",
      "present_in_code": true,
      "severity": "required"
    },
    {
      "kind": "missing_term",
      "term": "solve",
      "present_in_code": false,
      "severity": "required"
    },
    {
      "kind": "extra_code_terms",
      "terms": [
        "carry",
        "def",
        "initial_state",
        "innovation",
        "jnp",
        "kalman_filter_scan",
        "lax",
        "linalg",
        "ll_t",
        "obs",
        "p_filt",
        "p_pred",
        "params",
        "return",
        "scan",
        "sign",
        "slogdet",
        "step",
        "x_filt",
        "x_pred"
      ],
      "severity": "audit_only"
    }
  ],
  "extra_in_code": [
    "carry",
    "def",
    "initial_state",
    "innovation",
    "jnp",
    "kalman_filter_scan",
... truncated for report ...
```

Chapter 11

Workflow 4: Audit a Derivation

The v2 audit workflow combines parser policy, typed obligations, route decisions, shape diagnostics, numeric suggestions, backend evidence, and verification-boundary text. It is the most appropriate colleague-facing surface when a user asks whether a labeled mathematical statement is actually certified.

Listing 11.1: Derivation audit output

```
Command: python -m mathdevmcp.cli audit-derivation-v2-label eq:dept-state-space-likelihood --root <repo>/benchmarks/fixtures --summary-only
{
  "label": "eq:dept-state-space-likelihood",
  "doc_root": "<repo>/benchmarks/fixtures",
  "status": "unverified",
  "reason": "At least one obligation remains unverified or diagnostic-only.",
  "counts": {
    "total": 1,
    "verified": 0,
    "mismatch": 0,
    "unverified": 1,
    "inconclusive": 0
  },
  "high_priority_actions": [
    {
      "kind": "state_or_verify_missing_constraint",
      "target": "invertibility_required",
      "severity": "high",
      "reason": "Matrix inverse or solve notation requires an invertible or positive-definite operand.",
      "obligation_id": "obligation_1"
    },
    {
      "kind": "state_or_verify_missing_constraint",
      "target": "square_matrix_required",
      "severity": "high",
      "reason": "Determinant and log determinant require a square matrix operand.",
      "obligation_id": "obligation_1"
    },
    {
      "kind": "state_or_verify_missing_constraint",
      "target": "conformable_product_required",
      "severity": "high",
```

```
      "reason": "Transpose and matrix product notation require conformable dimensions.",
      "obligation_id": "obligation_1"
    },
    {
      "kind": "human_formalization_or_review",
      "severity": "high",
      "reason": "Typed obligation has missing assumptions or dimension constraints.",
      "obligation_id": "obligation_1"
    },
    {
      "kind": "logdet_domain_check",
      "target": "determinant/logdet operand",
      "severity": "high",
      "reason": "Log determinant requires square positive-definite or otherwise valid matrix input.",
      "obligation_id": "obligation_1"
    },
    {
      "kind": "linear_solve_residual_check",
      "target": "inverse/solve operand",
      "severity": "high",
      "reason": "Inverse notation should be checked with solve residuals and conditioning diagnostics.",
      "obligation_id": "obligation_1"
    }
  ],
  "obligations": [
    {
      "id": "obligation_1",
      "label": "eq:dept-state-space-likelihood",
      "status": "unverified",
      "reason": "The obligation has missing shape, dimension, or regularity constraints.",
      "evidence_kind": "diagnostic_bundle",
      "diagnostic_only": true,
      "verification_boundary": "The obligation has missing shape, dimension, or regularity constraints. A deterministic backend certificate is required before this can become verified.",
      "route": "human_review",
      "missing_constraints": [
        {
          "kind": "invertibility_required",
          "target": "inverse operand",
          "reason": "Matrix inverse or solve notation requires an invertible or positive-definite operand.",
          "status": "missing_assumption",
          "source": "unresolved_constructs"
        },
        {
          "kind": "square_matrix_required",
          "target": "determinant/logdet operand",
          "reason": "Determinant and log determinant require a square matrix operand.",
          "status": "missing_assumption",
          "source": "unresolved_constructs"
        },
... truncated for report ...
```

Chapter 12

Workflow 5: Build an Implementation Brief

The implementation brief bundles search, label selection, context extraction, consistency checking, and optional derivation checking into one object. This is the recommended tool when another agent needs a document-grounded starting point for code review or implementation.

Listing 12.1: Implementation brief output

```
Command: python -m mathdevmcp.cli implementation-brief state space likelihood <repo>/benchmarks/fixtures/doc_department_state_space_missing_solve.py --root <repo>/benchmarks/fixtures --required-terms logdet,solve --limit 2
{
  "query": "state space likelihood",
  "doc_root": "<repo>/benchmarks/fixtures",
  "code_path": "<repo>/benchmarks/fixtures/doc_department_state_space_missing_solve.py",
  "search_results": [
    {
      "kind": "equation",
      "name": "equation",
      "file": "doc_department_state_space.tex",
      "line_start": 21,
      "line_end": 23,
      "label": "eq:dept-state-space-likelihood",
      "title": null,
      "text": "\\begin{equation}\\label{eq:dept-state-space-likelihood}\n\\ell_t = -\\frac{1}{2}\\left(\\log\\det \\InnovCov + \\Innov' \\InnovCov^{-1}\\Innov\\right).\n\\end{equation}",
      "block_id": "doc_department_state_space.tex:21:equation:eq:dept-state-space-likelihood",
      "section_path": [
        "Sanitized state-space audit slice"
      ],
      "score": 3
    },
    {
      "kind": "section",
      "name": "Sanitized state-space audit slice",
      "file": "doc_department_state_space.tex",
      "line_start": 1,
```

```
      "line_end": 1,
      "label": null,
      "title": "Sanitized state-space audit slice",
      "text": "\\section{Sanitized state-space audit slice}",
      "block_id": "doc_department_state_space.tex:1:section:Sanitized-state-space-audit-slice",
      "section_path": [
        "Sanitized state-space audit slice"
      ],
      "score": 2
    }
  ],
  "selected_label": "eq:dept-state-space-likelihood",
  "status": "mismatch",
  "cache": {
    "path": null,
    "hit": false
  },
  "checks": {
    "consistency": {
      "status": "mismatch",
      "reason": "Some required document terms were not found in the code text.",
      "doc_terms": [
        "logdet",
        "solve"
      ],
      "code_terms": [
        "def",
        "kalman_filter_scan",
        "jnp",
        "lax",
        "params",
        "step",
        "carry",
        "obs",
        "x_pred",
        "p_pred",
        "innovation",
        "sign",
        "logdet",
        "linalg",
        "slogdet",
        "x_filt",
        "p_filt",
        "ll_t",
        "return",
        "scan",
        "initial_state"
      ],
      "missing_in_code": [
        "solve"
      ],
      "findings": [
        {
          "kind": "matched_term",
          "term": "logdet",
... truncated for report ...
```

Part IV

Case Studies

Chapter 13

Kalman State-Space Likelihood

The Kalman likelihood case checks whether implementation code carries the linear algebra structure described by the document. Missing solves, inverses, or log-determinant terms are important because they often indicate stale documentation or a numerically fragile implementation. MathDevMCP reports such cases as consistency findings rather than silently rationalizing them.

For a colleague, the useful behavior is fast localization. The tool identifies the label, extracts the surrounding text, compares the implementation file, and reports missing required terms. The next action is human review of the likelihood implementation and associated tests.

Chapter 14

HMC Leapfrog and Hamiltonian Flow

Hamiltonian Monte Carlo code is sensitive to sign conventions, gradients, Metropolis corrections, and leapfrog ordering. MathDevMCP does not certify convergence. It checks that documented Hamiltonian and leapfrog structures are visible in the implementation and routes uncertified claims to abstention when regularity or ergodicity assumptions are not present.

Chapter 15

Macro Filter Multi-File Corpus

The macro filter fixture exercises multi-file LaTeX inclusion, macro definitions, repeated notation, and a deliberately missing gain update. This is close to real colleague work: a main document includes a model file, macros are defined in one place, and code may omit a step that the exposition assumes.

Chapter 16

DSGE Euler Equation

The DSGE case checks Euler residual and stochastic discount factor structure. MathDevMCP helps a reviewer avoid claiming that a model is solved or calibrated unless the assumptions and implementation evidence are available. The expected abstention is part of the release quality signal.

Chapter 17

Stochastic Volatility Likelihood

The stochastic volatility case checks transition and likelihood labels. The important release behavior is conservative handling of latent-volatility prior assumptions and Jacobian-like omissions. A diagnostic may expose a missing term, but the tool should not overstate proof.

Chapter 18

SDE and PDE Numerics

The SDE/PDE numerics case covers Euler–Maruyama and residual/stability structure. A colleague can use the tool to locate the discretization formula, check whether code carries time-step and stability terms, and record a discretization-assumption review when the evidence is incomplete.

Chapter 19

ML and LLM Objective Functions

The ML objective case catches wrong-sign gradients and missing objective terms. This is useful for training and fine-tuning code where the prose can describe a loss while the implementation evolves quickly. MathDevMCP provides a structured way to ask whether the documented loss and gradient are visible in code.

Chapter 20

Bayesian ELBO and Variational Inference

The ELBO case checks objective, expectation, entropy, and reparameterization structure. The release behavior emphasizes abstention around variational-family assumptions and support conditions. This keeps the agent from treating a surface-level match as a full variational proof.

Chapter 21

Computational Physics MCMC

The computational physics case checks acceptance ratios, Hamiltonian energy, gradients, and leapfrog-like operations. The tool can surface a missing Metropolis correction or energy term, but it deliberately leaves convergence and ergodicity claims for expert review.

Chapter 22

Private Corpus Validation

The private-corpus workflow validates external manifests without committing private material. Normal output redacts private paths. The generated evidence below uses a sanitized external corpus under the same release machinery that a real private department manifest would use.

Listing 22.1: Private corpus validation summary

```
Command: MATHDEVMCP_PRIVATE_CORPUS_MANIFEST=<redacted-private-manifest> scripts/validate_private_corpus.sh "$PWD"
Status: consistent
Reason: Private corpus manifest is configured and satisfies release-gate privacy checks.
Private paths redacted: True
Findings: 0
Parser reports: [('private_dsge_macro_finance_euler', 'selected_for_proof_audit'), ('private_stochastic_volatility_likelihood', 'selected_for_proof_audit'), ('private_sde_pde_numerics', 'selected_for_proof_audit'), ('private_ml_llm_objective', 'selected_for_proof_audit'), ('private_bayesian_elbo_vi', 'selected_for_proof_audit'), ('private_computational_physics_mcmc', 'selected_for_proof_audit')]
```

Part V

Security, Operations, and Maintenance

Chapter 23

Security and Privacy

Private documents must stay outside the repository. A populated private manifest may contain sensitive paths and should remain external. Normal MathDevMCP reports redact private document roots and code roots. Governance validation rejects private paths inside the checkout. Evidence bundles for review should be stored outside git unless they are deliberately reduced to safe report snippets.

Chapter 24

Operations

The normal operator sequence is:

```
PYTHONPATH=src python -m mathdevmcp.cli doctor
scripts/backend_env_doctor.sh "$PWD"
MATHDEVMCP_REQUIRE_LATEXML=1 scripts/validate_latexml_backend.sh "$PWD"
PYTHONPATH=src python -m mathdevmcp.cli benchmark-gate --root "$PWD"
PYTHONPATH=src python -m mathdevmcp.cli release-readiness --root "$PWD" --profile base
```

For full release validation, add:

```
export MATHDEVMCP_PRIVATE_CORPUS_MANIFEST=/secure/local/path/corpus.json
scripts/validate_private_corpus.sh "$PWD"
PYTHONPATH=src python -m mathdevmcp.cli release-readiness --root "$PWD" --profile full
```

Chapter 25

Maintainer Guide

Maintain release behavior by changing library code first, then CLI/MCP wrappers, then docs and evidence. When adding a new benchmark category, update the case builder, runner, summary tests, release report evidence, and release policy if the category becomes mandatory. When adding a new private corpus entry, keep the entry outside git and check redaction before sharing output.

The highest-risk modules are:

- `benchmarks.py`, for benchmark totals and gate behavior;
- `release_policy.py`, for profile blockers and caveats;
- `release_corpus.py`, for privacy redaction and manifest shape;
- `parser_policy.py`, for proof-audit parser routing;
- `proof_audit_v2.py`, for verification-boundary reporting;
- `mcp_facade.py` and `cli.py`, for public tool surfaces.

Chapter 26

Limitations and Accepted Boundaries

The product is conservative by design. It is useful because it knows when to stop. It does not convert every mathematical statement to a theorem prover. It does not guarantee semantic equivalence between arbitrary prose and code. It does not make private data safe to commit. It does not make numeric diagnostics into proofs. These boundaries should remain explicit in future releases.

Part VI

Detailed Release Manual

Chapter 27

End-to-End Colleague Scenario

This chapter walks through a realistic review day. A colleague has a document claim about a state-space likelihood and a Python implementation that is being edited quickly before a deadline. The colleague does not want a polished essay; they want to know where the claim lives, whether the code visibly contains the required numerical operations, whether a derivation can be certified, and what should be reviewed by a human.

27.1 Step 1: locate the candidate claim

The agent begins with `search-latex`. This avoids relying on a language model's memory of the repository. The search result gives labels and local context. The colleague can inspect those labels directly or ask the agent to use the top result. If no label is found, the workflow stops as *inconclusive* rather than inventing a source.

27.2 Step 2: extract neighborhood context

The label neighborhood gives surrounding prose and assumptions. This step matters because many mathematical claims are not self-contained. A likelihood equation may rely on notation introduced one paragraph above, while a stability condition may depend on a boundary condition or discretization convention.

27.3 Step 3: compare to code

The comparison step asks for concrete required terms such as `logdet`, `solve`, `gradient`, or `stability`. A missing term is not automatically a proof of a bug, but it is a precise review signal. The output includes the selected label, findings, and status, so the agent can explain what was checked.

27.4 Step 4: audit the derivation boundary

The v2 audit report is where MathDevMCP becomes most valuable for disciplined agent work. It records parser policy, typed diagnostics, shape diagnostics, numeric diagnostic suggestions, backend attempts, and a verification boundary. That boundary text is essential: it tells the colleague whether the output is a certificate, a diagnostic, or an abstention.

27.5 Step 5: produce an implementation brief

For handoff, the implementation brief bundles the key evidence. A teammate can read one object rather than rerun every command. This is especially useful when the agent is asked to create a code-review note, a pull-request comment, or a maintenance ticket.

Chapter 28

Release Gate Interpretation

The release gate should be read as a contract, not a vibe check. A green base profile means public fixtures, governance, release corpus metadata, parser policy, and benchmark cases pass. It does not mean every optional backend or private corpus is available. A green full profile means the strict optional evidence has also been supplied. This separation lets the team use MathDevMCP productively on ordinary machines while still preserving a stricter bar for industrial release.

28.1 Blockers

Blockers represent release conditions that must be fixed or explicitly scoped out. Examples include benchmark failure, parser policy failure, governance validation failure, missing strict LaTeXML evidence in the LaTeXML profile, missing backend evidence in the backend profile, and missing private manifest evidence in the private-corpus or full profiles.

28.2 Caveats

Caveats are not ignored. They are tracked because a release may be useful while still carrying environmental or evidence limitations. A dirty worktree is a caveat because generated reports should name the actual commit. Missing private corpus evidence is a low-severity caveat in the base profile, but a blocker in the private-corpus and full profiles.

28.3 Expected abstentions

Expected abstentions protect the product from false confidence. A benchmark can pass because the tool correctly abstained in a case where certification would be inappropriate. This is different from a tolerated failure budget. The current benchmark policy requires all cases to pass.

Chapter 29

Private Corpus Operating Model

Private corpus validation is designed for teams that cannot commit research documents, code paths, or real local directory structures. The populated manifest lives outside git. The release validator reads it locally, uses unredacted paths internally to parse documents, and emits redacted reports.

29.1 External sanitized corpus

For autonomous release validation in this workspace, the execution created an external sanitized corpus under a temporary directory. It includes six release domains: DSGE macro-finance, stochastic volatility, SDE/PDE numerics, ML/LLM objectives, Bayesian ELBO/VI, and computational-physics MCMC. The generated manifest is not committed. The report includes only redacted summaries.

29.2 Real private corpus substitution

A real deployment should replace the sanitized manifest with an approved external manifest. The same validator and full profile commands apply. The operator should verify that every release-gated entry has expected labels, parser backends, expected abstentions or false-confidence seeds, and external document roots. If a real private corpus covers fewer domains than the sanitized release corpus, the release report should say exactly that.

29.3 Failure modes

Important private-corpus failure modes are:

- missing manifest path;

- invalid JSON;
- malformed entry fields;
- private path inside the repository checkout;
- missing document root or code root;
- release-gated entry with no expected labels;
- release-gated entry with no parser backend;
- parser policy not selected for proof audit.

Chapter 30

Backend Environment Operating Model

Backend tools are intentionally not all installed into the main Python environment. This is a release-quality design choice. LeanDojo can bring a dependency stack that conflicts with unrelated document or ML packages. The backend env checks therefore prove that the optional backend can be invoked without requiring the main working environment to import every optional module.

30.1 Lean

Lean is detected as an executable, usually through the user-local `elan` proxy. The release pins the intended toolchain through `MATHDEVMCP_LEAN_TOOLCHAIN` for MathDevMCP subprocesses. This avoids changing the user's global Lean default.

30.2 LeanDojo

LeanDojo is checked as a Python module inside `mathdevmcp-backends`. A plain `doctor` command in the active environment may report LeanDojo as unavailable. The backend env doctor is the authoritative check for the backend profile.

30.3 LaTeXML

LaTeXML is a system executable, not a Python dependency. The strict LaTeXML profile requires it to preserve expected labels on the benchmark corpus. The current evidence shows label preservation and source provenance. Environment recognition differs from the lightweight parser, so parser policy still chooses the current parser for proof-audit routing.

Chapter 31

Code Architecture for Maintainers

The implementation should be maintained by preserving the boundary between library logic, CLI rendering, MCP wrappers, scripts, and documentation.

31.1 Library logic

Library functions build structured dictionaries with contract metadata. They should be deterministic for the same input files and environment. New behavior should be added in library code first and exercised by direct tests.

31.2 CLI

The CLI should remain a thin argument parser. It should call library functions, print JSON, and return meaningful exit codes. It should not contain hidden business logic that bypasses the tested library surface.

31.3 MCP facade

The MCP facade should wrap the same library functions. Tool names are part of the colleague-facing API. If a tool is renamed, keep a compatibility alias or document a breaking change.

31.4 Scripts

Scripts should orchestrate repeatable workflows. They should set `PYTHONPATH` explicitly, avoid shell string evaluation for backend commands, and preserve redaction. Scripts that create generated evidence should write to ignored or explicitly requested output directories.

Chapter 32

Adding a New Domain

To add a new mathematical domain, follow this release path:

1. Add a small public or sanitized fixture with at least one labeled mathematical block.
2. Add paired code showing the relevant operations or a seeded mismatch.
3. Extend AST operation extraction only if the new operation is genuinely needed.
4. Add benchmark cases for parser preservation, AST operation coverage, and expected abstention behavior.
5. Add release corpus metadata.
6. Add a report case study and generated evidence snippet if the domain is release-facing.
7. Update private manifest templates only with placeholder paths.

This path keeps new domains measurable. It also prevents the product from accumulating impressive-sounding workflows that do not have regression tests.

Chapter 33

Adding a New Backend

A new backend should enter the system as optional evidence. The correct path is to add a doctor capability, an isolated invocation strategy if needed, timeout protection, a clear result contract, and tests for unavailable, timeout, inconclusive, mismatch, and certifying behavior. Only after that should the backend be used in proof-audit routing or release profiles.

33.1 Backend result discipline

Backend results should distinguish:

- backend unavailable;
- input not encodable;
- timeout;
- diagnostic-only evidence;
- counterexample or mismatch;
- deterministic certificate.

The last item is intentionally rare. Most mathematical development tools are useful because they reduce search space and expose gaps, not because they automatically certify every claim.

Chapter 34

Release Evidence Maintenance

The evidence snippets in this report are generated by `scripts/generate_release_report_evidence.sh`. Regenerate them after changing release policy, parser policy, benchmark cases, private corpus validation, or user-facing workflow examples. Do not hand-edit generated evidence unless the file is explicitly converted into narrative documentation.

34.1 Evidence hygiene

After regeneration, run a path leak scan. The scan should check for real private roots, local manifest filenames, and repository-local absolute paths. The committed snippets may contain `<repo>`, `<redacted-private-path>`, and `<redacted-private-manifest>`.

34.2 Evidence reproducibility

Generated snippets should name the command that produced them. A colleague reading the report should be able to rerun the command in the repository and understand why the local result may differ if the worktree is dirty, optional backends are missing, or a private manifest is not configured.

Chapter 35

Risk Register

35.1 Risk: overclaiming certification

The most important product risk is false confidence. Mitigation: keep verification-boundary text in proof-audit reports, preserve diagnostic-only flags, and require benchmark cases where abstention is the correct outcome.

35.2 Risk: private data leakage

Private roots and manifests are sensitive. Mitigation: keep populated manifests outside git, redact normal reports, reject private paths inside the checkout, and scan generated docs before commit.

35.3 Risk: backend dependency conflict

Optional backends can conflict with the main environment. Mitigation: use `mathdevmcp-backends`, backend-specific doctor checks, and explicit environment variables rather than broad installation into the base env.

35.4 Risk: benchmark drift

Benchmark totals and category meanings can drift as the project grows. Mitigation: update tests, reset memo, and release report together whenever a benchmark case is added or removed.

35.5 Risk: large-module entropy

The codebase has several large modules. Mitigation: keep public facades stable, extract pure helper modules when it improves locality, and run focused tests after each extraction.

Appendices

Appendix A

Release Corpus Summary

Listing A.1: Release corpus validation summary

```
Command: PYTHONPATH=src python -m mathdevmcp.cli validate-release-
    corpus --root "$PWD/benchmarks/fixtures"
Status: consistent
Findings: 0
Entries: 22
Private manifest status: loaded
Private paths redacted: True
```

Appendix B

Benchmark Gate JSON Excerpt

Listing B.1: Benchmark gate JSON excerpt


```json
{
  "ok": true,
  "passed": true,
  "total": 41,
  "passed_count": 41,
  "failed_count": 0,
  "summary": {
    "by_category": {
      "consistency": {
        "total": 5,
        "passed": 5,
        "expected_abstentions": 0
      },
      "derivation": {
        "total": 6,
        "passed": 6,
        "expected_abstentions": 5
      },
      "workflow": {
        "total": 3,
        "passed": 3,
        "expected_abstentions": 1
      },
      "proof_audit": {
        "total": 3,
        "passed": 3,
        "expected_abstentions": 1
      },
      "proof_audit_v2": {
        "total": 3,
        "passed": 3,
        "expected_abstentions": 1
      },
      "kalman_recursion": {
        "total": 2,
        "passed": 2,
        "expected_abstentions": 1
      },
      "parser_corpus": {
        "total": 3,
```

```
        "passed": 3,
        "expected_abstentions": 0
      },
      "ast_corpus": {
        "total": 11,
        "passed": 11,
        "expected_abstentions": 0
      },
      "typed_ir": {
        "total": 2,
        "passed": 2,
        "expected_abstentions": 2
      },
      "industrial_review": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "release_corpus": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 0
      },
      "release_policy": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 0
      }
    },
    "by_focus": {
      "status_regression": {
        "total": 2,
        "passed": 2,
        "expected_abstentions": 0
      },
      "provenance_correctness": {
        "total": 2,
        "passed": 2,
        "expected_abstentions": 0
      },
      "realistic_fixture": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 0
      },
      "abstention_quality": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "false_confidence_control": {
        "total": 6,
        "passed": 6,
        "expected_abstentions": 0
      },
      "realistic_abstention": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
```

```
      },
      "multilabel_provenance": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "long_document_provenance": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "repeat_label_stability": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "workflow_contract": {
        "total": 3,
        "passed": 3,
        "expected_abstentions": 1
      },
      "proof_audit_routing": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 0
      },
      "proof_audit_abstention": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "release_spine_verified": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 0
      },
      "release_spine_abstention": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "ast_recursion_abstention": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "realistic_parser_provenance": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 0
      },
      "multifile_macro_parser_provenance": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 0
      },
      "sanitized_domain_parser_provenance": {
        "total": 1,
        "passed": 1,
```

```
        "expected_abstentions": 0
      },
      "realistic_ast_operation_coverage": {
        "total": 3,
        "passed": 3,
        "expected_abstentions": 0
      },
      "sanitized_domain_ast_operation_coverage": {
        "total": 6,
        "passed": 6,
        "expected_abstentions": 0
      },
      "typed_dimension_diagnostics": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "typed_stochastic_diagnostics": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "agent_review_packet": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "release_corpus_manifest": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 0
      },
      "release_readiness_contract": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 0
      }
    },
    "expected_abstentions": 12
  },
  "policy": {
    "name": "all_benchmarks_must_pass",
    "required_pass_rate": 1.0,
    "allow_category_failures": {},
    "description": "Every benchmark case must pass; no category-specific failure budget is currently allowed."
  },
  "metadata": {
    "schema_version": "1.0",
    "contract": "benchmark_gate"
  }
}
```

Appendix C

Parser Policy JSON Excerpt

Listing C.1: Parser policy JSON excerpt


```json
{
  "status": "selected_for_proof_audit",
  "reason": "Current parser preserves expected labels and line provenance for this
      corpus.",
  "selected_backend": "current",
  "blocking_findings": [],
  "caveats": [],
  "backend_roles": [
    {
      "backend": "current",
      "role": "selected_for_proof_audit",
      "status": "parsed"
    }
  ],
  "benchmark_report": {
    "ok": true,
    "results": [
      {
        "backend": "current",
        "status": "parsed",
        "reason": "Current lightweight parser completed.",
        "labels_found": 67,
        "environments_found": 67,
        "align_like_found": 4,
        "provenance_quality": "line",
        "runtime_seconds": 0.005802565996418707,
        "labels": [
          "assump:dept-hmc-regularity",
          "assump:dept-state-space-guards",
          "assump:macro-filter-dimensions",
          "eq:dept-acceptance-ratio",
          "eq:dept-elbo",
          "eq:dept-euler-equation",
          "eq:dept-euler-maruyama",
          "eq:dept-hamiltonian-flow",
          "eq:dept-hmc-hamiltonian",
          "eq:dept-hmc-leapfrog",
          "eq:dept-log-posterior",
          "eq:dept-ml-gradient",
          "eq:dept-ml-loss",
```

```json
      "eq:dept-particle-ess",
      "eq:dept-particle-normalization",
      "eq:dept-pde-stability",
      "eq:dept-reparameterization-gradient",
      "eq:dept-sdf",
      "eq:dept-state-space-likelihood",
      "eq:dept-state-space-recursion",
      "eq:dept-sv-likelihood",
      "eq:dept-sv-transition",
      "eq:domain-hamiltonian-commute",
      "eq:domain-hamiltonian-missing-assumption",
      "eq:domain-kalman-unsupported",
      "eq:kalman-innovation-covariance",
      "eq:kalman-innovation-score-local",
      "eq:kalman-prediction-covariance",
      "eq:kalman-score-contribution",
      "eq:longdoc-innovation-covariance",
      "eq:longdoc-innovation-residual",
      "eq:longdoc-observed-information",
      "eq:longdoc-score-contribution",
      "eq:longdoc-smoothing-gain",
      "eq:longdoc-transition-covariance",
      "eq:macro-filter-innovation",
      "eq:macro-filter-likelihood",
      "eq:macro-filter-transition",
      "eq:proof-audit-ambiguous",
      "eq:proof-audit-chain",
      "eq:proof-audit-false",
      "eq:proof-audit-kalman",
      "eq:proof-audit-single",
      "eq:repeat-kalman-covariance-01",
      "eq:repeat-kalman-covariance-02",
      "eq:repeat-kalman-covariance-03",
      "eq:repeat-kalman-covariance-04",
      "eq:repeat-kalman-covariance-05",
      "eq:repeat-kalman-covariance-06",
      "eq:repeat-kalman-covariance-07",
      "eq:repeat-kalman-covariance-08",
      "eq:repeat-kalman-smoothing-01",
      "eq:repeat-kalman-smoothing-02",
      "eq:repeat-kalman-smoothing-03",
      "eq:repeat-kalman-smoothing-04",
      "eq:repeat-kalman-smoothing-05",
      "eq:repeat-kalman-smoothing-06",
      "eq:repeat-kalman-smoothing-07",
      "eq:repeat-kalman-smoothing-08",
      "eq:repeat-kalman-target-score",
      "eq:transport-density-step",
      "prop:hamiltonian-energy",
      "prop:macro-filter-repeat-notation",
      "prop:proof-audit-nearby",
      "prop:transport-implementation",
      "prop:transport-logdet",
      "prop:transport-mismatch"
    ],
    "quality_checks": {
      "label_preservation": true,
      "environment_recognition": true,
      "align_detection": true,
```

```
          "provenance_available": true
        },
        "details": {
          "n_blocks": 100,
          "n_labels": 67,
          "environment_types": [
            "align",
            "assumption",
            "equation",
            "proposition",
            "section",
            "subsection"
          ],
          "environment_type_counts": {
            "align": 4,
            "assumption": 3,
            "equation": 54,
            "proposition": 6,
            "section": 32,
            "subsection": 1
          },
          "tex_files_scanned": [
            "doc_consistency_bad.tex",
            "doc_consistency_context.tex",
            "doc_consistency_good.tex",
            "doc_department_bayesian_hmc.tex",
            "doc_department_particle_filter.tex",
            "doc_department_state_space.tex",
            "doc_derivation_chain.tex",
            "doc_domain_formalization.tex",
            "doc_longdoc_kalman_retrieval.tex",
            "doc_macro_filter_main.tex",
            "doc_macro_filter_model.tex",
            "doc_multilabel_kalman_chain.tex",
            "doc_proof_audit.tex",
            "doc_realistic_hamiltonian.tex",
            "doc_realistic_kalman_hessian.tex",
            "doc_repeat_label_kalman_scale.tex",
            "doc_sanitized_bayesian_elbo_vi.tex",
            "doc_sanitized_computational_physics_mcmc.tex",
            "doc_sanitized_dsge_macro_finance.tex",
            "doc_sanitized_ml_llm_objective.tex",
            "doc_sanitized_sde_pde_numerics.tex",
            "doc_sanitized_stochastic_volatility.tex"
          ],
          "per_file_metrics": {
            "doc_consistency_bad.tex": {
              "labels": [
                "prop:transport-mismatch"
              ],
              "label_count": 1,
              "environment_count": 1,
              "environment_type_counts": {
                "proposition": 1
              },
              "include_targets": [],
              "macro_definition_count": 0
            },
            "doc_consistency_context.tex": {
```

```
              "labels": [
                "prop:transport-implementation"
              ],
              "label_count": 1,
              "environment_count": 1,
              "environment_type_counts": {
                "proposition": 1
              },
              "include_targets": [],
              "macro_definition_count": 0
            },
            "doc_consistency_good.tex": {
              "labels": [
                "prop:transport-logdet"
              ],
              "label_count": 1,
              "environment_count": 1,
              "environment_type_counts": {
                "proposition": 1
              },
              "include_targets": [],
              "macro_definition_count": 0
            },
            "doc_department_bayesian_hmc.tex": {
              "labels": [
                "assump:dept-hmc-regularity",
                "eq:dept-hmc-hamiltonian",
                "eq:dept-hmc-leapfrog",
                "eq:dept-log-posterior"
              ],
              "label_count": 4,
              "environment_count": 4,
              "environment_type_counts": {
                "align": 1,
                "assumption": 1,
                "equation": 2
              },
              "include_targets": [],
              "macro_definition_count": 1
            },
            "doc_department_particle_filter.tex": {
              "labels": [
                "eq:dept-particle-ess",
                "eq:dept-particle-normalization"
              ],
              "label_count": 2,
              "environment_count": 2,
              "environment_type_counts": {
                "equation": 2
              },
              "include_targets": [],
              "macro_definition_count": 0
            },
            "doc_department_state_space.tex": {
              "labels": [
                "assump:dept-state-space-guards",
                "eq:dept-state-space-likelihood",
                "eq:dept-state-space-recursion"
              ],
```

```
            "label_count": 3,
            "environment_count": 3,
            "environment_type_counts": {
              "align": 1,
... truncated for release report appendix ...
```

Appendix D

Release Corpus JSON Excerpt

Listing D.1: Release corpus JSON excerpt


```
{
  "status": "consistent",
  "reason": "Release corpus manifest satisfies privacy and release-gate checks.",
  "findings": [],
  "manifest": {
    "entries": [
      {
        "id": "kalman_state_space_extended",
        "domain": "kalman_state_space",
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
        "code_roots": [
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-state-space-recursion",
          "eq:dept-state-space-likelihood"
        ],
        "expected_operations": [
          "logdet",
          "inverse_or_solve",
          "quadratic_form",
          "prediction_update",
          "covariance_update"
        ],
        "expected_abstentions": [
          "eq:dept-state-space-likelihood"
        ],
        "seeded_false_confidence_cases": [
          "doc_department_state_space_missing_solve.py"
        ],
        "required_parser_backends": [
          "current"
        ],
        "release_gate_enabled": true,
        "notes": "Existing sanitized state-space fixtures exercise parser, AST, typed IR, and proof-audit v2 abstention."
      },
      {
        "id": "hmc_nuts_leapfrog",
```

```
      "domain": "hmc_nuts",
      "privacy_class": "public_fixture",
      "document_root": "<repo>/benchmarks/fixtures",
      "code_roots": [
        "<repo>/benchmarks/fixtures"
      ],
      "expected_labels": [
        "eq:dept-hmc-leapfrog",
        "eq:dept-hmc-hamiltonian"
      ],
      "expected_operations": [
        "gradient",
        "leapfrog_update",
        "hamiltonian_energy"
      ],
      "expected_abstentions": [
        "eq:dept-hmc-leapfrog"
      ],
      "seeded_false_confidence_cases": [
        "hmc_missing_gradient_seed"
      ],
      "required_parser_backends": [
        "current"
      ],
      "release_gate_enabled": true,
      "notes": "Sanitized HMC fixtures cover leapfrog and Hamiltonian structure without claiming full NUTS correctness."
    },
    {
      "id": "particle_filter_logsumexp",
      "domain": "particle_filter",
      "privacy_class": "public_fixture",
      "document_root": "<repo>/benchmarks/fixtures",
      "code_roots": [
        "<repo>/benchmarks/fixtures"
      ],
      "expected_labels": [
        "eq:dept-particle-normalization",
        "eq:dept-particle-ess"
      ],
      "expected_operations": [
        "logsumexp",
        "particle_normalization"
      ],
      "expected_abstentions": [
        "particle_filter_weight_degeneracy_review"
      ],
      "seeded_false_confidence_cases": [
        "particle_filter_missing_logsumexp_seed"
      ],
      "required_parser_backends": [
        "current"
      ],
      "release_gate_enabled": true,
      "notes": "Public sanitized particle-filter fixture covers log-sum-exp normalization and ESS diagnostics."
    },
    {
      "id": "dsge_macro_finance_euler_public",
```

```
        "domain": "dsge_macro_finance",
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
        "code_roots": [
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-euler-equation",
          "eq:dept-sdf"
        ],
        "expected_operations": [
          "expectation",
          "euler_residual"
        ],
        "expected_abstentions": [
          "private_calibration_assumption_review"
        ],
        "seeded_false_confidence_cases": [
          "missing_discount_factor_seed"
        ],
        "required_parser_backends": [
          "current"
        ],
        "release_gate_enabled": true,
        "notes": "Public synthetic macro-finance fixture for Euler residual and stochastic discount factor structure."
      },
      {
        "id": "stochastic_volatility_likelihood_public",
        "domain": "stochastic_volatility",
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
        "code_roots": [
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-sv-transition",
          "eq:dept-sv-likelihood"
        ],
        "expected_operations": [
          "innovation_update",
          "posterior_or_likelihood"
        ],
        "expected_abstentions": [
          "latent_volatility_prior_review"
        ],
        "seeded_false_confidence_cases": [
          "missing_jacobian_seed"
        ],
        "required_parser_backends": [
          "current"
        ],
        "release_gate_enabled": true,
        "notes": "Public synthetic stochastic-volatility transition and likelihood fixture."
      },
      {
        "id": "sde_pde_numerics_public",
        "domain": "sde_pde_numerics",
```

```
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
        "code_roots": [
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-euler-maruyama",
          "eq:dept-pde-stability"
        ],
        "expected_operations": [
          "time_step_update",
          "stability_condition"
        ],
        "expected_abstentions": [
          "discretization_assumption_review"
        ],
        "seeded_false_confidence_cases": [
          "unstable_step_size_seed"
        ],
        "required_parser_backends": [
          "current"
        ],
        "release_gate_enabled": true,
        "notes": "Public synthetic SDE/PDE discretization and stability fixture."
      },
      {
        "id": "ml_llm_objective_public",
        "domain": "ml_llm_objective",
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
        "code_roots": [
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-ml-loss",
          "eq:dept-ml-gradient"
        ],
        "expected_operations": [
          "gradient",
          "posterior_or_likelihood"
        ],
        "expected_abstentions": [
          "data_pipeline_assumption_review"
        ],
        "seeded_false_confidence_cases": [
          "wrong_sign_gradient_seed"
        ],
        "required_parser_backends": [
          "current"
        ],
        "release_gate_enabled": true,
        "notes": "Public synthetic ML objective and gradient sign fixture."
      },
      {
        "id": "bayesian_elbo_vi_public",
        "domain": "bayesian_elbo_vi",
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
        "code_roots": [
```

```
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-elbo",
          "eq:dept-reparameterization-gradient"
        ],
        "expected_operations": [
          "elbo_objective",
... truncated for release report appendix ...
```

Appendix E

Full Release Readiness JSON Excerpt

Listing E.1: Full release readiness JSON excerpt


```
{
  "status": "ready_with_caveats",
  "reason": "Release gates passed with documented caveats.",
  "profile": "full",
  "profile_policy_version": "2026-04-caveat-closure",
  "required_capabilities": [
    "benchmark_gate",
    "current_parser_policy",
    "governance_validation",
    "release_corpus_manifest",
    "lean_dojo_backend_env",
    "latexml",
    "private_corpus_manifest"
  ],
  "optional_capabilities": [],
  "evidence_commands": [
    "PYTHONPATH=src python -m mathdevmcp.cli doctor",
    "PYTHONPATH=src python -m mathdevmcp.cli release-readiness --root \"$PWD\" --profile full",
    "PYTHONPATH=src python -m mathdevmcp.cli validate-release-corpus --root \"$PWD/benchmarks/fixtures\"",
    "scripts/release_smoke.sh \"$PWD\"",
    "scripts/backend_env_doctor.sh \"$PWD\"",
    "scripts/validate_backend_install.sh \"$PWD\"",
    "MATHDEVMCP_REQUIRE_LATEXML=1 scripts/validate_latexml_backend.sh \"$PWD\"",
    "MATHDEVMCP_PRIVATE_CORPUS_MANIFEST=/secure/local/path/corpus.json scripts/validate_private_corpus.sh \"$PWD\""
  ],
  "package_version": "editable_or_uninstalled",
  "git_commit": "2f49963",
  "dirty_worktree": true,
  "benchmark_gate": {
    "ok": true,
    "passed": true,
    "total": 40,
    "passed_count": 40,
    "failed_count": 0,
    "summary": {
      "by_category": {
        "consistency": {
```

```
        "total": 5,
        "passed": 5,
        "expected_abstentions": 0
      },
      "derivation": {
        "total": 6,
        "passed": 6,
        "expected_abstentions": 5
      },
      "workflow": {
        "total": 3,
        "passed": 3,
        "expected_abstentions": 1
      },
      "proof_audit": {
        "total": 3,
        "passed": 3,
        "expected_abstentions": 1
      },
      "proof_audit_v2": {
        "total": 3,
        "passed": 3,
        "expected_abstentions": 1
      },
      "kalman_recursion": {
        "total": 2,
        "passed": 2,
        "expected_abstentions": 1
      },
      "parser_corpus": {
        "total": 3,
        "passed": 3,
        "expected_abstentions": 0
      },
      "ast_corpus": {
        "total": 11,
        "passed": 11,
        "expected_abstentions": 0
      },
      "typed_ir": {
        "total": 2,
        "passed": 2,
        "expected_abstentions": 2
      },
      "industrial_review": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 1
      },
      "release_corpus": {
        "total": 1,
        "passed": 1,
        "expected_abstentions": 0
      }
    },
    "by_focus": {
      "status_regression": {
        "total": 2,
        "passed": 2,
```

```
      "expected_abstentions": 0
    },
    "provenance_correctness": {
      "total": 2,
      "passed": 2,
      "expected_abstentions": 0
    },
    "realistic_fixture": {
      "total": 1,
      "passed": 1,
      "expected_abstentions": 0
    },
    "abstention_quality": {
      "total": 1,
      "passed": 1,
      "expected_abstentions": 1
    },
    "false_confidence_control": {
      "total": 6,
      "passed": 6,
      "expected_abstentions": 0
    },
    "realistic_abstention": {
      "total": 1,
      "passed": 1,
      "expected_abstentions": 1
    },
    "multilabel_provenance": {
      "total": 1,
      "passed": 1,
      "expected_abstentions": 1
    },
    "long_document_provenance": {
      "total": 1,
      "passed": 1,
      "expected_abstentions": 1
    },
    "repeat_label_stability": {
      "total": 1,
      "passed": 1,
      "expected_abstentions": 1
    },
    "workflow_contract": {
      "total": 3,
      "passed": 3,
      "expected_abstentions": 1
    },
    "proof_audit_routing": {
      "total": 1,
      "passed": 1,
      "expected_abstentions": 0
    },
    "proof_audit_abstention": {
      "total": 1,
      "passed": 1,
      "expected_abstentions": 1
    },
    "release_spine_verified": {
      "total": 1,
```

```
          "passed": 1,
          "expected_abstentions": 0
        },
        "release_spine_abstention": {
          "total": 1,
          "passed": 1,
          "expected_abstentions": 1
        },
        "ast_recursion_abstention": {
          "total": 1,
          "passed": 1,
          "expected_abstentions": 1
        },
        "realistic_parser_provenance": {
          "total": 1,
          "passed": 1,
          "expected_abstentions": 0
        },
        "multifile_macro_parser_provenance": {
          "total": 1,
          "passed": 1,
          "expected_abstentions": 0
        },
        "sanitized_domain_parser_provenance": {
          "total": 1,
          "passed": 1,
          "expected_abstentions": 0
        },
        "realistic_ast_operation_coverage": {
          "total": 3,
          "passed": 3,
          "expected_abstentions": 0
        },
        "sanitized_domain_ast_operation_coverage": {
          "total": 6,
          "passed": 6,
          "expected_abstentions": 0
        },
        "typed_dimension_diagnostics": {
          "total": 1,
          "passed": 1,
          "expected_abstentions": 1
        },
        "typed_stochastic_diagnostics": {
          "total": 1,
          "passed": 1,
          "expected_abstentions": 1
        },
        "agent_review_packet": {
          "total": 1,
          "passed": 1,
          "expected_abstentions": 1
        },
        "release_corpus_manifest": {
          "total": 1,
          "passed": 1,
          "expected_abstentions": 0
        }
      },
```

```
      "expected_abstentions": 12
    },
    "policy": {
      "name": "all_benchmarks_must_pass",
      "required_pass_rate": 1.0,
      "allow_category_failures": {},
      "description": "Every benchmark case must pass; no category-specific failure budget is currently allowed."
    },
    "metadata": {
      "schema_version": "1.0",
      "contract": "benchmark_gate"
    }
  },
  "doctor_summary": {
    "ok": true,
    "python": {
      "executable": "/home/chakwong/miniconda3/envs/tfgpu/bin/python",
      "version": "3.11.15",
      "prefix": "/home/chakwong/miniconda3/envs/tfgpu",
      "path_head": [
        "/home/chakwong/bin",
        "/home/chakwong/.codex/tmp/arg0/codex-arg0U2SIb1",
        "/home/chakwong/.vscode-server/bin/10c8e557c8b9f9ed0a87f61f1c9a44bde731c409/bin/remote-cli",
        "/home/chakwong/miniconda3/envs/tfgpu/bin",
        "/home/chakwong/bin"
      ]
    },
    "capabilities": {
      "latexml": {
        "name": "latexml",
        "available": true,
        "kind": "executable",
        "path": "/usr/bin/latexml",
        "version": "latexml (LaTeXML version 0.8.6)",
        "status": "available",
        "detail": "available"
      },
      "pandoc": {
        "name": "pandoc",
        "available": true,
        "kind": "executable",
        "path": "/usr/bin/pandoc",
        "version": "pandoc 2.9.2.1",
        "status": "available",
        "detail": "available"
      },
      "lean": {
        "name": "lean",
        "available": true,
        "kind": "executable",
        "path": "/home/chakwong/.elan/bin/lean",
        "version": "Lean (version 4.20.0, x86_64-unknown-linux-gnu, commit 77cfc4d1a4f6, Release)",
        "status": "available",
        "detail": "available"
      },
      "sage": {
```

```
        "name": "sage",
        "available": true,
        "kind": "executable",
        "path": "/usr/bin/sage",
        "version": "SageMath version 9.5, Release Date: 2022-01-30",
        "status": "available",
        "detail": "available"
      },
      "lean_dojo": {
        "name": "lean_dojo",
... truncated for release report appendix ...
```

Appendix F

Private Corpus JSON Excerpt

Listing F.1: Private corpus JSON excerpt


```json
{
  "status": "consistent",
  "reason": "Private corpus manifest is configured and satisfies release-gate privacy checks.",
  "private_paths_redacted": true,
  "manifest": {
    "entries": [
      {
        "id": "kalman_state_space_extended",
        "domain": "kalman_state_space",
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
        "code_roots": [
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-state-space-recursion",
          "eq:dept-state-space-likelihood"
        ],
        "expected_operations": [
          "logdet",
          "inverse_or_solve",
          "quadratic_form",
          "prediction_update",
          "covariance_update"
        ],
        "expected_abstentions": [
          "eq:dept-state-space-likelihood"
        ],
        "seeded_false_confidence_cases": [
          "doc_department_state_space_missing_solve.py"
        ],
        "required_parser_backends": [
          "current"
        ],
        "release_gate_enabled": true,
        "notes": "Existing sanitized state-space fixtures exercise parser, AST, typed IR, and proof-audit v2 abstention."
      },
      {
```

```
      "id": "hmc_nuts_leapfrog",
      "domain": "hmc_nuts",
      "privacy_class": "public_fixture",
      "document_root": "<repo>/benchmarks/fixtures",
      "code_roots": [
        "<repo>/benchmarks/fixtures"
      ],
      "expected_labels": [
        "eq:dept-hmc-leapfrog",
        "eq:dept-hmc-hamiltonian"
      ],
      "expected_operations": [
        "gradient",
        "leapfrog_update",
        "hamiltonian_energy"
      ],
      "expected_abstentions": [
        "eq:dept-hmc-leapfrog"
      ],
      "seeded_false_confidence_cases": [
        "hmc_missing_gradient_seed"
      ],
      "required_parser_backends": [
        "current"
      ],
      "release_gate_enabled": true,
      "notes": "Sanitized HMC fixtures cover leapfrog and Hamiltonian structure without claiming full NUTS correctness."
    },
    {
      "id": "particle_filter_logsumexp",
      "domain": "particle_filter",
      "privacy_class": "public_fixture",
      "document_root": "<repo>/benchmarks/fixtures",
      "code_roots": [
        "<repo>/benchmarks/fixtures"
      ],
      "expected_labels": [
        "eq:dept-particle-normalization",
        "eq:dept-particle-ess"
      ],
      "expected_operations": [
        "logsumexp",
        "particle_normalization"
      ],
      "expected_abstentions": [
        "particle_filter_weight_degeneracy_review"
      ],
      "seeded_false_confidence_cases": [
        "particle_filter_missing_logsumexp_seed"
      ],
      "required_parser_backends": [
        "current"
      ],
      "release_gate_enabled": true,
      "notes": "Public sanitized particle-filter fixture covers log-sum-exp normalization and ESS diagnostics."
    },
    {
```

```
      "id": "dsge_macro_finance_euler_public",
      "domain": "dsge_macro_finance",
      "privacy_class": "public_fixture",
      "document_root": "<repo>/benchmarks/fixtures",
      "code_roots": [
        "<repo>/benchmarks/fixtures"
      ],
      "expected_labels": [
        "eq:dept-euler-equation",
        "eq:dept-sdf"
      ],
      "expected_operations": [
        "expectation",
        "euler_residual"
      ],
      "expected_abstentions": [
        "private_calibration_assumption_review"
      ],
      "seeded_false_confidence_cases": [
        "missing_discount_factor_seed"
      ],
      "required_parser_backends": [
        "current"
      ],
      "release_gate_enabled": true,
      "notes": "Public synthetic macro-finance fixture for Euler residual and stochastic discount factor structure."
    },
    {
      "id": "stochastic_volatility_likelihood_public",
      "domain": "stochastic_volatility",
      "privacy_class": "public_fixture",
      "document_root": "<repo>/benchmarks/fixtures",
      "code_roots": [
        "<repo>/benchmarks/fixtures"
      ],
      "expected_labels": [
        "eq:dept-sv-transition",
        "eq:dept-sv-likelihood"
      ],
      "expected_operations": [
        "innovation_update",
        "posterior_or_likelihood"
      ],
      "expected_abstentions": [
        "latent_volatility_prior_review"
      ],
      "seeded_false_confidence_cases": [
        "missing_jacobian_seed"
      ],
      "required_parser_backends": [
        "current"
      ],
      "release_gate_enabled": true,
      "notes": "Public synthetic stochastic-volatility transition and likelihood fixture."
    },
    {
      "id": "sde_pde_numerics_public",
```

```
        "domain": "sde_pde_numerics",
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
        "code_roots": [
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-euler-maruyama",
          "eq:dept-pde-stability"
        ],
        "expected_operations": [
          "time_step_update",
          "stability_condition"
        ],
        "expected_abstentions": [
          "discretization_assumption_review"
        ],
        "seeded_false_confidence_cases": [
          "unstable_step_size_seed"
        ],
        "required_parser_backends": [
          "current"
        ],
        "release_gate_enabled": true,
        "notes": "Public synthetic SDE/PDE discretization and stability fixture."
      },
      {
        "id": "ml_llm_objective_public",
        "domain": "ml_llm_objective",
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
        "code_roots": [
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-ml-loss",
          "eq:dept-ml-gradient"
        ],
        "expected_operations": [
          "gradient",
          "posterior_or_likelihood"
        ],
        "expected_abstentions": [
          "data_pipeline_assumption_review"
        ],
        "seeded_false_confidence_cases": [
          "wrong_sign_gradient_seed"
        ],
        "required_parser_backends": [
          "current"
        ],
        "release_gate_enabled": true,
        "notes": "Public synthetic ML objective and gradient sign fixture."
      },
      {
        "id": "bayesian_elbo_vi_public",
        "domain": "bayesian_elbo_vi",
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
```

```
        "code_roots": [
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-elbo",
          "eq:dept-reparameterization-gradient"
        ],
        "expected_operations": [
          "elbo_objective",
          "reparameterization_gradient",
          "expectation"
        ],
        "expected_abstentions": [
          "variational_family_assumption_review"
        ],
        "seeded_false_confidence_cases": [
          "missing_entropy_seed"
        ],
        "required_parser_backends": [
          "current"
        ],
        "release_gate_enabled": true,
        "notes": "Public synthetic ELBO and reparameterization-gradient fixture."
      },
      {
        "id": "computational_physics_mcmc_public",
        "domain": "computational_physics_mcmc",
        "privacy_class": "public_fixture",
        "document_root": "<repo>/benchmarks/fixtures",
        "code_roots": [
          "<repo>/benchmarks/fixtures"
        ],
        "expected_labels": [
          "eq:dept-acceptance-ratio",
          "eq:dept-hamiltonian-flow"
        ],
        "expected_operations": [
          "acceptance_ratio",
          "hamiltonian_energy",
          "gradient"
        ],
        "expected_abstentions": [
          "ergodicity_assumption_review"
        ],
        "seeded_false_confidence_cases": [
          "missing_metropolis_correction_seed"
        ],
        "required_parser_backends": [
          "current"
... truncated for release report appendix ...
```

Appendix G

Evidence Index

Listing G.1: Generated evidence index

```
Generated release report evidence snippets:
doctor-summary.txt
benchmark-gate-summary.txt
parser-policy-summary.txt
release-corpus-summary.txt
release-readiness-full-summary.txt
private-corpus-summary.txt
workflow-search.txt
workflow-context.txt
workflow-compare.txt
workflow-audit.txt
workflow-brief.txt
benchmark-gate-json-excerpt.txt
parser-policy-json-excerpt.txt
release-corpus-json-excerpt.txt
release-readiness-full-json-excerpt.txt
private-corpus-json-excerpt.txt
```

Appendix H

Command Reference

Command	Purpose
`doctor`	Report runtime capabilities and optional backend status.
`search-latex`	Search a LaTeX source tree for relevant context.
`extract-latex-neighborhood`	Extract paragraph context around a label.
`compare-label-code`	Compare labeled document terms to code.
`audit-derivation-v2-label`	Build typed proof-audit evidence for a label.
`implementation-brief`	Bundle search, context, and code checks.
`benchmark-gate`	Run the release benchmark gate.
`validate-release-corpus`	Validate public and configured private corpus metadata.
`release-readiness`	Produce a profile-specific release report.

Appendix I

Private Manifest Fields

Field	Meaning
`id`	Stable entry identifier.
`domain`	Mathematical or colleague workflow domain.
`privacy_class`	`private_external` or `private_sanitized_external`.
`document_root`	External LaTeX/document root.
`code_roots`	External code roots associated with the documents.
`expected_labels`	Labels the parser must preserve.
`expected_operations`	Mathematical operations expected in code or AST evidence.
`expected_abstentions`	Known cases where conservative abstention is expected.
`seeded_false_confidence_cases`	False-confidence seeds for review.
`required_parser_backends`	Parser backends required for the entry.
`release_gate_enabled`	Whether the entry is mandatory for private profile evidence.
`notes`	Human-readable release note.